Jekyll + ITCSS Action Plan

Goal

Make the website architecture more consistent, unified, and simplified while preserving your current naming where possible.

This plan follows the decisions from our discussion:

- keep changes targeted
- prefer Jekyll-first simplicity
- follow ITCSS cleanly
- avoid renaming unless the rename improves clarity
- do not create abstractions before the HTML structure is stable

High-level strategy

The work should happen in this order:

1. Fix broken user-facing behavior first.
2. Clean up naming inconsistencies in `page.html`, `projects.html`, and `index.html`.
3. Add the highest-confidence shared objects.
4. Simplify repeated HTML structures before extracting more objects.
5. Fill the missing `6-layouts` layer.
6. Move mis-layered selectors out of `3-base`.
7. Only create optional objects like `o-card` if the final CSS duplication proves they are necessary.

Phase 1 — Fix broken behavior first

1.1 Fix the mobile menu toggle

Goal

Make the hamburger menu in `mobile-menu.html` work.

Files involved

- `_includes/header.html`
- `_includes/mobile-menu.html`
- `assets/js/common.js`

What to do

- Use the existing hooks already present in the HTML:
  - `data-nav-toggle`
  - `data-mobile-menu`
  - `data-nav-close`
  - `aria-controls="mobile-menu"`
  - `aria-expanded`
  - `hidden`
- Add the menu open/close logic to `assets/js/common.js` instead of creating a separate JS file.

Behavior to implement

- On hamburger click:
  - open the mobile menu
  - remove `hidden`
  - set `aria-expanded="true"`
  - optionally add an open state class like `.is-open`
- On close button click:
  - close the mobile menu
  - add `hidden`
  - set `aria-expanded="false"`
  - remove the open state class
- Optional but recommended:
  - close on `Escape`
  - close when clicking a mobile nav link
  - lock body scroll while menu is open

Why this comes first

This is the only clearly broken interaction. Fixing it improves the real user experience immediately.

Phase 2 — Rename inconsistent HTML classes

2.1 Update `_layouts/page.html`

Goal

Align the page component naming with the file and layout naming.

Current problem

`page.html` uses layout classes named `l-page*` but component classes named `c-article*`.
That split is inconsistent.

Rename plan

Rename:

- `.c-article` → `.c-page`
- `.c-article__media` → `.c-page__media`
- `.c-article__image` → `.c-page__image`
- `.c-article__title` → `.c-page__title`
- `.c-article__subtitle` → `.c-page__subtitle`
- `.c-article__meta` → `.c-page__meta`
- `.c-article__author` → `.c-page__author`
- `.c-article__meta-sep` → `.c-page__meta-sep`
- `.c-article__date` → `.c-page__date`
- `.c-article__divider` → `.c-page__divider`
- `.c-tag` → `.c-page__tag`

Also add

- apply `.o-prose` to the page body wrapper

Recommended wrapper pattern:

- layout owns placement: `l-page__body`

- object owns long-form typography: `o-prose`

Why

- better naming harmony
- easier SCSS organization
- cleaner mental model for `page.html`

2.2 Update `projects.html`

Goal
Make the projects archive naming page-specific and clearer.

Rename plan

Layout classes

Rename to a project-specific layout namespace:

- `.l-page-header` → replace with a projects layout structure
- use:
  - `.l-projects`
  - `.l-projects__header`
  - `.l-projects__filters`
  - `.l-projects__archive`

Component classes

Rename:

- `.c-page-header` → `.c-projects-header`
- `.c-page-header__title` → `.c-projects-header__title`
- `.c-page-header__subtitle` → `.c-projects-header__subtitle`

Keep:

- `.c-projects-filter`
- `.c-projects-filter__inner`
- `.c-projects-filter__chip`
- `.c-projects-archive`

Why

- the page is specifically the projects archive
- the names become more self-explanatory
- this gives `6-layouts` a clean page-specific structure

2.3 Update `index.html`

Goal

Rename generic section layout classes to homepage-specific layout classes.

Rename plan

Rename:

- `.l-section` → `.l-homepage-section`
- `.l-section--projects` → `.l-homepage-section--projects`

Keep component classes grouped under the home namespace:

- `.c-home-projects__header`
- `.c-home-projects__title`
- `.c-home-projects__actions`
- `.c-home-projects__button`

Do not create

- `.o-home-section`

Why

This is a layout concern, not an object concern.

Phase 3 — Add the highest-confidence shared objects

3.1 Create `4-objects/_o-tag.scss`

Goal

Unify tag styling across page types while preserving context-specific class names.

Applies to

- `.c-post__tag`
- `.c-post-card__tag`
- `.c-page__tag`
- possible future tag archive tag elements

What `o-tag` should own

- inline-flex alignment
- padding
- border-radius
- border
- background
- default font family
- default font weight
- default font size
- line-height
- transition
- baseline hover and focus behavior for linked tags
- optional wrapping behavior

What component classes can still override

- size
- local spacing
- context-specific colors if needed

Why

This is the strongest reuse case in the project.

3.2 Create `4-objects/_o-prose.scss`

Goal

Create a reusable object for long-form content areas.

Use in

- `post.html`
- `page.html`
- future long-form content pages

What `o-prose` should own

- readable content width if needed
- spacing between headings, paragraphs, lists, blockquotes, tables, images, code, and embeds
- typography rhythm
- content-specific link treatment if needed
- list indentation and list marker behavior for prose content

What it should not own

- outer page width containment
- page shell spacing
- component-specific header styles

Difference from `o-container`

- `o-container` controls page-level horizontal containment
- `o-prose` controls long-form content rhythm and readability

Difference from `3-base`

- `3-base` styles raw elements globally
- `o-prose` styles long-form elements inside a specific content context

Why

This is a clean ITCSS object and avoids stuffing content rules into `3-base`.

3.3 Create `4-objects/_o-action-cluster.scss`

Goal

Unify repeated inline action groups.

Applies to

- `post-share.html`
- social link groupings where useful

What `o-action-cluster` should own

- flex or inline-flex arrangement
- wrapping behavior
- gap
- alignment

What it should not own

- icon sizing
- button or link visuals
- color
- borders

Why

This is a true generic arrangement pattern and fits ITCSS very well.

Phase 4 — Simplify metadata before extracting a metadata object

4.1 Revise the metadata markup in `post-card.html` and `post.html`

Goal

Reduce unnecessary structure before introducing a shared metadata object.

Files involved

- `_includes/post-card.html`
- `_layouts/post.html`

What to review

Shared metadata content currently includes:

- author avatar
- author name
- date
- reading time
- reading time icon

`post.html` additionally includes:

- share actions in the metadata row area
- a separator element between date and reading time

Specific action

- remove the date/read-time separator from `post.html`
- simplify the nested wrappers where possible
- make both metadata structures more parallel without forcing them to be identical

Why

The metadata object should be based on stable, simplified markup, not current duplication.

4.2 Create `4-objects/_o-post-metadata.scss`

Goal

Create a small, honest metadata object after the markup is simplified.

Preferred name

- `o-post-metadata`

Recommended first version

Use only the pieces that match your real content model:

- `.o-post-metadata`
- `.o-post-metadata__avatar`
- `.o-post-metadata__content`
- `.o-post-metadata__author`
- `.o-post-metadata__date`
- `.o-post-metadata__read-time`
- `.o-post-metadata__icon`

Do not overbuild yet

Avoid introducing extra abstraction such as:

- `__primary`
- `__secondary`
- `__item`
- `__body`

unless the simplified HTML clearly needs them.

Do not create yet

- a shared `post-meta.html` include

Why

The post and post-card metadata are similar, but not identical enough yet to justify shared markup.

Phase 5 — Fill the missing `6-layouts` layer

Your layout layer is the biggest structural gap right now.

5.1 Create `6-layouts/_l-homepage.scss`

Goal
Own homepage-specific layout spacing and section structure.

Suggested scope

- `.l-homepage-section`
- `.l-homepage-section--projects`

What this file should own

- vertical spacing between homepage sections
- section placement
- home-specific spacing modifiers

5.2 Create `6-layouts/_l-projects.scss`

Goal

Own page-level layout structure for `projects.html`.

Suggested scope

- `.l-projects`
- `.l-projects__header`
- `.l-projects__filters`
- `.l-projects__archive`

What this file should own

- spacing between major page zones
- page-level layout relationships
- archive/filter/header separation

5.3 Create `6-layouts/_l-page.scss`

Goal

Own layout structure for generic content pages.

Suggested scope

- `.l-page`
- `.l-page__header`
- `.l-page__tags`
- `.l-page__body`

What this file should own

- spacing between page header, tag row, and content body
- page-level layout placement only

5.4 Create `6-layouts/_l-tag-page.scss`

Goal

Own the tag archive page layout.

Suggested scope

- `.l-tag-page`
- `.l-tag-page__header`
- `.l-tag-page__title`

What this file should own

- spacing
- structural layout for the tag archive page

Phase 6 — Add the missing components

6.1 Create `5-components/_c-page.scss`

Goal

Style the renamed `c-page*` component in `_layouts/page.html`.

Suggested scope

- `.c-page`
- `.c-page__media`
- `.c-page__image`
- `.c-page__title`
- `.c-page__subtitle`
- `.c-page__meta`
- `.c-page__author`
- `.c-page__date`
- `.c-page__divider`

- `.c-page__tag`

What this file should own

- visual styles specific to generic pages
- page image treatment
- page-specific typography for title/subtitle/meta

6.2 Create `5-components/_c-projects.scss`

Goal
Keep all projects-page-specific visual components together.

Suggested scope

- `.c-projects-header`
- `.c-projects-header__title`
- `.c-projects-header__subtitle`
- `.c-projects-filter`
- `.c-projects-filter__inner`
- `.c-projects-filter__chip`
- `.c-projects-archive`

What this file should own

- projects page header look
- filter chip visuals
- projects archive visual section styling

6.3 Create `5-components/_c-home.scss`

Goal
Keep homepage-specific visual components together.

Suggested scope

- `.c-home-projects__header`
- `.c-home-projects__title`

- `.c-home-projects__actions`
- `.c-home-projects__button`

What this file should own

- homepage section header look
- homepage CTA/button alignment

6.4 Optional: create `5-components/_c-tag-page.scss`

Goal
Only create this if the tag archive needs component-level styling beyond layout.

Create only if needed

Examples:

- special tag archive title styling
- archive-specific labels or counts
- page-specific visual treatment that does not belong in layout

Phase 7 — Clean up `3-base` and move mis-layered selectors

7.1 Fix mis-layered image/media selectors

Goal
Remove component selectors from base styles.

Problem

Your base layer currently includes image/media rules tied to component selectors.
That breaks ITCSS separation.

Correct split

`3-base/_media.scss`

Keep only true element defaults such as:

- `img`
- `picture`
- `figure`
- maybe `video`

`4-objects/_o-media-frame.scss` (optional)

Use only if shared media wrappers are real.

Possible scope:

- generic overflow handling
- shared border-radius
- image fill behavior
- object-fit wrapper behavior

`5-components/*`

Keep component-specific image/media styling in component files.

Examples:

- hero image treatment
- post cover image treatment
- page image treatment
- post-card image treatment
- post-nav media treatment

Recommendation

Do not place component selectors in `3-base`.
If you need shared media structure, use `4-objects/_o-media-frame.scss`.

7.2 Fix list marker and list indentation behavior

Goal

Fix bullet and numbered lists so markers sit properly within readable content blocks.

Split the fix correctly

In `3-base`

Add sane global list defaults:

- default `padding-inline-start`
- default margins for `ul`, `ol`, and nested lists
- spacing between sibling `li` items

In `o-prose`

Add content-specific improvements:

- typography-aware list spacing
- refined marker alignment
- nested list rhythm for long-form content
- optional marker color adjustments

Why

- base should set safe defaults site-wide
- `o-prose` should refine lists for article/page reading contexts

Phase 8 — Decide later on optional objects

8.1 Decide later on `4-objects/_o-card.scss`

Current stance

Do not create this immediately.

Why

You still need to confirm whether `.c-post-card` and `.c-post-nav__item` truly share the same surface rules after the rest of the cleanup.

If you do create it later

Keep it narrowly defined as a generic surface object.

Suggested scope

- background
- border
- border-radius
- box-shadow
- hover or focus lift
- transition
- optional overflow clipping

Do not let it own

- content spacing
- card media layout
- typography
- metadata structure
- tag placement

ITCSS rationale

This is valid only if it stays a generic structural shell and does not become a pseudo-component.

8.2 Decide later on `4-objects/_o-media-frame.scss`

Current stance

Optional.

Create it only if multiple components truly share the same media wrapper logic.

Why

Some media behavior can stay inside component files until duplication is clear.

Suggested file list after the refactor

4-objects

Create or consider creating:

- `_o-tag.scss`
- `_o-post-metadata.scss`
- `_o-prose.scss`
- `_o-action-cluster.scss`
- `_o-card.scss` only later if justified
- `_o-media-frame.scss` only if justified

5-components

Create:

- `_c-page.scss`
- `_c-projects.scss`
- `_c-home.scss`
- `_c-tag-page.scss` only if needed

6-layouts

Create:

- `_l-homepage.scss`
- `_l-projects.scss`
- `_l-page.scss`
- `_l-tag-page.scss`

Recommended implementation order

Priority 1

Fix the mobile menu in `assets/js/common.js` .

Priority 2

Rename and clean up `page.html`:

- `c-article*` → `c-page*`
- `c-tag` → `c-page__tag`
- add `o-prose` to the content body

Priority 3

Rename and clean up `projects.html`:

- introduce `l-projects*`
- rename `c-page-header*` → `c-projects-header*`

Priority 4

Rename and clean up `index.html`:

- `l-section*` → `l-homepage-section*`

Priority 5

Implement the strongest shared objects:

- `_o-tag.scss`
- `_o-prose.scss`
- `_o-action-cluster.scss`

Priority 6

Simplify metadata markup in:

- `_includes/post-card.html`
- `_layouts/post.html`

Then implement:

- `_o-post-metadata.scss`

Priority 7

Create the missing layout files:

- `_l-homepage.scss`
- `_l-projects.scss`
- `_l-page.scss`
- `_l-tag-page.scss`

Priority 8

Create the missing component files:

- `_c-page.scss`
- `_c-projects.scss`
- `_c-home.scss`
- optional `_c-tag-page.scss`

Priority 9

Clean up `3-base`:

- move mis-layered image/media selectors out of base
- split list fixes between base and `o-prose`

Priority 10

Re-evaluate whether `_o-card.scss` and `_o-media-frame.scss` are truly needed.

Decision summary

Implement now

- mobile menu JS in `assets/js/common.js`
- rename `c-article*` → `c-page*`
- rename `c-tag` → `c-page__tag`
- rename projects layout/component classes
- rename homepage section layout classes

- `o-tag`
- `o-prose`
- `o-action-cluster`
- later in this cycle: `o-post-metadata`
- missing `6-layouts` files
- missing `5-components` files

Implement later only if needed

- `o-card`
- `o-media-frame`
- `_c-tag-page.scss`

Do not do right now

- do not create a shared `post-meta.html` include
- do not place component selectors in `3-base`
- do not create `o-home-section`

Final note

This plan keeps the architecture disciplined without over-engineering it.

The most important principle for the next pass is:

first stabilize naming and HTML structure, then extract shared objects only where reuse is clearly real.